

Warteliste und Belegungskalender

Organisation: Sie arbeiten im Zweierteam mit den Rollen A und B im Repository aus Blatt 1 und 2. Wo eine Aufgabe die Rollen trennt, arbeitet jede Person in einem eigenen Branch (a-..., b-...) und mergt nach umbau-zugangsregel.

Abgabe: Tag abgabe-3 auf main, danach ein ZIP des Projektordners **mit** dem Ordner .git über Moodle/APA. Frist: [OFFEN: Datum]. Testat 3 und die Vorführung verwenden die Klassennamen dieses Blatts.

Sie schreiben den Java-Code dieses Blatts ohne KI-Werkzeug und bedienen Git selbst, Befehl für Befehl. Im Testat und in der Vorführung erklären Sie jede Zeile und jeden Befehl.

Grundlage ist freiraum-start-3.zip aus Moodle: der Referenzstand nach Blatt 2 und neu die Klasse Beispieldaten. Alle Räume, Gruppen und Zeiten darin sind frei erfunden.

Erste Aufgabe (20 Minuten): Entscheiden Sie gemeinsam, ob Sie Ihren eigenen Stand aus Blatt 2 weiterführen oder den Startstand 3 übernehmen. Beispieldaten.java brauchen Sie in beiden Fällen. Spielen Sie das ZIP in einem Branch start-3 ein, wie es die README.md im ZIP beschreibt, und lesen Sie vor dem Commit mit `git diff --staged`, was sich gegenüber Ihrem Stand ändert. Mergen Sie start-3 in main, legen Sie den Branch umbau-zugangsregel an und pushen Sie ihn. Starten Sie PruefungRaumtypen: Die letzte Zeile lautet 24 OK, 0 FEHLER.

Zweite Aufgabe (50 Minuten): In Blatt 2 überschreibt Labor die Methode buchen und lehnt jede Buchung ohne Einweisung ab. Ihre Vertragsprobe in NOTIZEN.md hält fest, wo ein Aufrufer, der nur Raum kennt, davon überrascht wird. Die Regel wandert jetzt aus der Unterklasse in ein eigenes Objekt:

```
public interface Zugangsregel {  
  
    boolean erlaubt(Buchung b);  
  
    /** Grund, der gemeldet wird, wenn erlaubt false liefert. */  
    String grund();  
}
```

Committen Sie Zugangsregel.java zuerst gemeinsam auf umbau-zugangsregel. Erst danach trennen sich die Rollen.

- (A) OhneBeschraenkung erlaubt jede Buchung, Grund keine Beschränkung. NurMitEinweisung erlaubt nur Buchungen, für die hatEinweisung() true liefert, Grund nur mit Einweisung buchbar.
- (B) Raum erhält das Attribut `private final Zugangsregel regel` und einen zweiten Konstruktor `Raum(String kennung, int plaetze, Zugangsregel regel)`, der null als Regel mit einer `IllegalArgumentException` abweist. Der bisherige Konstruktor ruft den neuen mit `new OhneBeschraenkung()` auf. buchen fragt als Erstes `regel.erlaubt(b)`. Labor übergibt `new NurMitEinweisung()` an `super(...)` und überschreibt buchen nicht mehr; `maxDauerMinuten()` und `toString()` bleiben.

Nach dem Merge beider Branches läuft PruefungRaumtypen weiter ohne FEHLER. Schreiben Sie in NOTIZEN.md unter „Blatt 3“ Ihre Entscheidung in drei Sätzen: Welche Lösung gilt jetzt für das Labor, nach welchem Kriterium, und welchen Preis zahlen Sie dafür („Wir kaufen ... und bezahlen mit ...“)? Beziehen Sie sich auf Ihre Vertragsprobe aus Blatt 2.

Dritte Aufgabe (30 Minuten): Auf false folgt bisher nur „Mögliche Gründe: ...“. Schreiben Sie BuchungAbgelehntException extends Exception mit einem Konstruktor, der den Grund an super weitergibt, und der

Methode `String getGrund()`. Ziehen Sie alle Prüfungen aus `buchen` in eine neue Methode `public void pruefeZulassung(Buchung b) throws BuchungAbgelehntException`, die bei der ersten verletzten Bedingung wirft. `buchen` wird zu `public void buchen(Buchung b) throws BuchungAbgelehntException`, ruft `pruefeZulassung` auf und trägt die Buchung danach ein. Die Warteliste der vierten Aufgabe braucht `pruefeZulassung`, um zu prüfen, ohne zu buchen.

Der Grund nennt Raum und Anlass, zum Beispiel UX-LAB: nur mit Einweisung buchbar, LR-1 hat 12 Plätze, angefragt sind 14 Personen, LR-1 erlaubt höchstens 180 Minuten, angefragt sind 190 oder LR-1 ist Di 10:00–11:30 schon belegt (Team Blau).

Kompilieren Sie, bevor Sie das Terminal anpassen, und lesen Sie die Meldung an der Aufrufstelle in `Buchungsterminal`:

```
error: unreported exception BuchungAbgelehntException; must be caught or declared to be thrown
```

Bei deutscher Systemsprache erscheint sie auf Deutsch. Fangen Sie die Exception im Terminal und geben Sie Nicht gebucht: mit dem Grund aus. `PruefungRaum` und `PruefungRaumtypen` kompilieren jetzt nicht mehr: Wo sie false erwartet haben, prüfen sie ab jetzt, dass die Exception kommt und welchen Grund sie trägt.

Zum Schluss eine Probe: Ersetzen Sie den catch-Block im Terminal kurz durch `catch (Exception e) {}`, buchen Sie UX-LAB ohne Einweisung und lassen Sie sich danach die Belegung zeigen. Notieren Sie in `NOTIZEN.md`, was eine Person am Terminal jetzt sieht und was sie daraus schließt. Bauen Sie den catch-Block zurück.

Vierte Aufgabe (70 Minuten): Ist ein Zeitfenster belegt, soll sich eine Gruppe anstellen können. Bauen Sie dafür eine eigene einfach verkettete Liste, ohne Klassen aus `java.util`. Der Knoten:

```
/** Knoten der Warteliste. */
class Eintrag {

    Buchung buchung;
    Eintrag naechster;

    Eintrag(Buchung buchung) {
        this.buchung = buchung;
    }
}
```

Warteliste hält nur `private Eintrag kopf`. Zeichnen Sie jede Methode vor dem Schreiben für drei Lagen auf Papier: leere Liste, ein Eintrag, drei Einträge. Markieren Sie jeden Zeiger, der sich ändert.

- (A) `void hintenAnstellen(Buchung b)`, `Buchung vorneEntnehmen()` (liefert bei leerer Liste null) und `int laenge()`.
- (B) `boolean entfernen(String gruppe)` entfernt den ersten Eintrag dieser Gruppe und liefert false, wenn sie nicht wartet. `int position(String gruppe)` zählt ab 1 und liefert -1 für eine Gruppe, die nicht wartet. `toString()` liefert die Gruppen in Reihenfolge, etwa [Team B, Team C].

Danach gemeinsam in Raum: ein Attribut `private final Warteliste warteliste = new Warteliste()`, die Methode `int aufwarteliste(Buchung b)`, die die Position zurückgibt, und `boolean stornieren(int nummer)`. `stornieren` entfernt die Buchung mit dieser Nummer, schließt die Lücke im Array und lässt anschließend die erste Buchung der Warteliste nachrücken, für die `pruefeZulassung` nicht wirft; die übrigen Einträge behalten ihre Reihenfolge. `belegungAusgeben()` zeigt eine nicht leere Warteliste mit an.

Im Terminal fragt buchen nach einer Ablehnung, bei der `raum.istFrei(...)` false liefert, Auf die Warteliste setzen `(j/n)?` und meldet die Position. Das Menü erhält den Punkt 4 Stornieren. Probieren Sie den Ablauf von Hand: zwei Gruppen für dasselbe Fenster in LR-1, die erste stornieren, Belegung ansehen.

Fünfte Aufgabe (30 Minuten): Jede Person prüft den Code der anderen. (A) schreibt in `PruefungWarteliste` die Fälle für entfernen, (B) die Fälle für `vorneEntnehmen` mit anschließendem `hintenAnstellen`. Grundlage ist die Testfall-Batterie aus T07B, übertragen auf die Warteliste:

- leere Liste
- ein Element, danach erneut anstellen
- erstes Element
- letztes Element, danach hinten anstellen
- nicht vorhandene Gruppe
- zweimal dieselbe Gruppe: Welcher Eintrag geht?

Jeder Fall ist ein `pruefe`-Aufruf mit Aufruffolge und erwartetem Ergebnis; der Name sagt, was geprüft wird. Danach die Gegenprobe: (B) löscht in `entfernen` den Sonderfall für das erste Element, (A) in `hintenAnstellen` den für die leere Liste. Die Batterie der anderen Person muss beide Eingriffe melden. Beim ersten meldet die Referenzbatterie drei FEHLER, beim zweiten bricht sie mit einer `NullPointerException` ab. Tragen Sie die erste Zeile Ihrer Meldung wörtlich in `NOTIZEN.md` ein und bauen Sie beide Eingriffe zurück.

Sechste Aufgabe (70 Minuten): Ein Raum kennt nur seine eigenen Buchungen. Der Belegungskalender ordnet die Buchungen aller Räume nach ihrer Startzeit, als binärer Suchbaum mit dem Schlüssel `tag * 1440 + beginn` (Montag 10:00 ergibt 2040). Der Knoten:

```
/** Knoten des Belegungskalenders. */
class KalenderKnoten {

    int schluessel;
    Buchung buchung;
    KalenderKnoten links;
    KalenderKnoten rechts;
}
```

Ergänzen Sie einen Konstruktor, der die Buchung übernimmt und den Schlüssel mit `Belegungskalender.schluessel(buchung)` berechnet. `Belegungskalender` hält `private KalenderKnoten wurzel`. Jede öffentliche Methode ruft eine private rekursive Methode mit dem Knoten als Parameter auf.

- (A) `static int schluessel(Buchung b)`, `void einfuegen(Buchung b)` (kleinerer Schlüssel links, gleicher oder größerer rechts), `boolean enthaeltStart(int schluessel)` und `Buchung frueheste()` (liefert bei leerem Kalender null).
- (B) `int zaehle()`, `int hoehe()` (leerer Kalender 0, ein Knoten 1) und `void ausgebenSortiert()`, `inorder`, eine Zeile je Buchung, früheste zuerst.

Schreiben Sie `PruefungKalender` mit diesen Fällen: leerer Kalender (`zaehle`, `hoehe`, `frueheste`); fünf Montagsbuchungen mit den Startzeiten 600, 480, 720, 540, 660 in dieser Reihenfolge (`zaehle` 5, `hoehe` 3, früheste um 08:00); dieselben Startzeiten aufsteigend eingefügt (`hoehe` 5); zwei Buchungen mit gleicher Startzeit (`zaehle` 2); alle Buchungen aus

Beispieldaten.woche() (zaehle 40). Wer bei gleichem Schlüssel nichts einfügt, verliert Buchungen: Die Referenz zählt dann für woche() 34 statt 40.

Siebte Aufgabe (30 Minuten): Schreiben Sie Entartung mit einer main, die einen Kalender mit Beispieldaten.woche() und einen zweiten mit Beispieldaten.semesterplan() füllt und für beide zaehle() und hoehe() ausgibt. Der Semesterplan enthält 11.830 Buchungen, chronologisch sortiert. Der Referenzlauf ergibt für woche() die Höhe 9. Wie der Lauf beim Semesterplan endet, hängt vom Rechner ab.

- **Das Programm läuft durch.** Tragen Sie beide Höhen in NOTIZEN.md ein. Starten Sie Entartung dann ein zweites Mal mit einem Stack von 1 MB: `java -Xss1m -cp out freiraum.Entartung` (in der IDE als VM-Option `-Xss1m`). Auf dem Referenzrechner (macOS) bricht dieser Lauf mit `java.lang.StackOverflowError` in zaehle ab.
- **Das Programm bricht mit `java.lang.StackOverflowError` ab.** Das ist unter Windows und Linux mit der Voreinstellung von 1 MB Stack zu erwarten [PRÜFEN: Laufprobe unter Windows]. Tragen Sie die Höhe von woche() ein und leiten Sie die Höhe des Semesterplans ohne Lauf her.

In beiden Fällen kopieren Sie die ersten drei Zeilen des Fehlerbilds wörtlich in NOTIZEN.md und beantworten dort drei Fragen: Warum ergibt der chronologische Semesterplan eine Kette? Wie viele Knoten besucht enthaeltStart für die letzte Buchung am Freitag, und wie viele wären es bei Höhe 76, der Höhe desselben Plans in zufälliger Reihenfolge im Referenzlauf? In welcher Methode läuft der Stack über, und warum reicht derselbe Code für woche()?

Laufen alle Prüfprogramme ohne FEHLER, mergen Sie umbau-zugangsregel in main, setzen Sie den Tag abgabe-3 und pushen Sie Branch und Tag.